

前言

01.学习概述

- 1.1 学习目标
- 1.2 前置知识

02.需求背景与技术选型

- 2.1 业务需求分析
- 2.2 技术挑战识别
- 2.3 技术选型
- 2.4 设计模式
- 2.5 性能指标

03.整体架构设计

- 3.1 五层架构全景图
- 3.2 数据流转路径
- 3.3 生命周期管理
- 3.4 架构的可扩展性

04.协调层 - VoiceIntercomManager

- 4.1 Manager的职责和定位
- 4.2 核心方法：start() 深度解析
- 4.3 关键设计解析

05.策略层 - 设备适配策略

- 5.1 策略接口设计 IDeviceStrategy
- 5.2 宇视设备策略 YuShiStrategy
- 5.3 科达设备策略 KedaStrategy
- 5.4 自研NVR策略 WdzNvrStrategy
- 5.5 策略工厂 DeviceStrategyFactory

06.核心组件层

- 6.1 多线程流水线设计
- 6.2 队列选型
- 6.3 ByteBufferPool 对象池
- 6.4 AudioProcessorImpl

05.深度思考

- 5.1 关键问题探究
- 5.2 设计对比

06.实践验证

- 6.1 行为验证代码
- 6.2 性能测试

07.应用场景

- 7.1 最佳实践
- 7.2 使用禁忌

08.总结提炼

- 8.1 核心收获
- 8.2 知识图谱
- 8.3 延伸思考

09.参考资料

其他介绍

- 01.关于我的博客

前言

学习要符合如下的标准化链条：了解概念->探究原理->深入思考->总结提炼->底层实现->延伸应用"

01.学习概述

- 学习主题:

- 知识类型:

- ☐ ☒ Android/
 - ☐ ☒ 01.基础组件与机制
 - ☐ ☒ 四大组件
 - ☐ ☒ IPC机制
 - ☐ ☒ 消息机制
 - ☐ ☒ 事件分发机制
 - ☐ ☒ View与渲染体系 (含Window、复杂控件、动画)
 - ☐ ☒ 存储与数据安全 (SharedPreferences/DataStore/Room/Scoped Storage)
 - ☐ ☒ 02. 架构与工程化
 - ☐ ☒ 架构模式 (MVC/MVP/MVVM/MVI)
 - ☐ ☒ 依赖注入 (Koin/Hilt/Dagger)
 - ☐ ☒ 路由与模块化 (ARouter、Navigation)
 - ☐ ☒ Gradle与构建优化
 - ☐ ☒ 插件化与动态化
 - ☐ ☒ 插桩与监控框架
 - ☐ ☒ 03.性能优化与故障诊断
 - ☐ ☒ ANR分析与优化
 - ☐ ☒ 启动耗时优化
 - ☐ ☒ 内存泄漏监控
 - ☐ ☒ 监控与诊断工具
 - ☐ ☒ 04.Jetpack与生态框架
 - ☐ ☒ Room
 - ☐ ☒ Paging
 - ☐ ☒ WorkManager
 - ☐ ☒ Compose
 - ☐ ☒ 05.Framework与系统机制
 - ☐ ☒ ActivityManagerService (含ANR触发机制)
 - ☐ ☒ Binder机制
- ☐ ☒ 音视频开发/
 - ☐ ☒ 01.基础知识
 - ☐ ☒ 02.OpenGL渲染视频
 - ☐ ☒ 03.FFmpeg音视频解码
- ☐ ☒ Java/
 - ☐ ☒ 01.基础知识
 - ☐ ☒ 02.集合框架

- ☐ ☒ 03.异常处理
- ☐ ☒ 04.多线程与并发
- ☐ ☒ 06.JVM
- ☐ ☒ Kotlin/
 - ☐ ☒ 01.基础语法
 - ☐ ☒ 02.高阶扩展
 - ☐ ☒ 03.协程和流
- ☐ ☒ Flutter/
 - ☐ ☒ 01.基础知识
 - ☐ ☒ Dart 语言基础
 - ☐ ☒ Widget 基础与生命周期
 - ☐ ☒ Flutter 基础组件
 - ☐ ☒ 布局与约束
 - ☐ ☒ 绘制与渲染体系
 - ☐ ☒ 状态管理
 - ☐ ☒ 事件处理与手势系统
 - ☐ ☒ 原生通信
 - ☐ ☒ 02.路由与导航
 - ☐ ☒ 03.性能优化与故障诊断
 - ☐ ☒ 04.异步编程
 - ☐ ☒ 05.项目经验与案例沉淀
- ☐ ☒ 自我管理/
 - ☐ ☒ 01.内观
- ☐ ☒ 项目经验/
 - ☐ ☒ 01.启动逻辑
 - ☐ ☒ 02.云值守
 - ☐ ☒ 03.智控平台
 - ☐ ☒ 04.视频巡店

- 学习来源：视频巡店语音对讲部分
- 重要程度：★★★★★
- 学习日期：2025.
- 记录人：@panruiqi

1.1 学习目标

- 了解概念->探究原理->深入思考->总结提炼->底层实现->延伸应用"

1.2 前置知识



02.需求背景与技术选型

2.1 业务需求分析

核心功能需求：

- 视频巡店场景下的实时语音对讲功能
 - 用户在App端通过摄像头进行语音对讲
 - 支持按住说话（Push-to-Talk）模式
 - 需要连接状态提示（连接中、已连接、断开）

多设备支持需求

- 项目需要对接**3种不同厂商**的设备，后续要考虑萤石等其他多种类型的设备
 - 之前每个客户他们自己店铺用的很多的摄像头设备，不能说用了我们的服务，他们厂商的设备就不能用了，所以要考虑兼容性和扩展性
 - 难点：每种设备的连接方式、数据格式、握手流程完全不同，如何用统一的API对接？比如：

设备类型	厂商	协议特点
WDZ_NVR	自研	HTTP模拟POST，简单直连
KEDA	科达	自定义TCP协议头
YUSHI	宇视	复杂握手协议（4步）
EZVIZ	萤石云	SDK方式（暂未实现）

非功能性要求

- 性能要求：
 - 端到端延迟 < 500ms
 - 内存占用稳定，无泄漏
 - 采样的数据发送丢弃率低于5%
- 可维护性要求
 - 代码职责清晰
 - 新增设备类型简单
 - 易于调试和监控

2.2 技术挑战识别

挑战1：设备协议差异巨大

- 宇视设备（最复杂）

- ```
// 握手流程（4步）
1. Server → Client: ReleaseRandomKey { random: "abc123..." }
2. Client → Server: RequireSession { key: encrypt(random, streamKey) }
3. Server → Client: SessionAck { session: "xxx" }
4. Server → Client: BridgeReady { ... }

// ⚠ 只有收到BridgeReady后才能发送音频数据！
```

- 科达设备（中等复杂）

- ```
// 连接成功后立即发送协议头
POST /postAudio?streamKey=xxx RS/1.1\r\n
Host: \r\n
Content-Length:\r\n
Session-Id:\r\n
Authentication:\r\n\r\n

// 然后直接发送音频数据
```

- 自研NVR（最简单）

- ```
// 模拟HTTP POST
POST /control/audio HTTP/1.1\r\n
Host: 192.168.1.100:8000\r\n
Content-Length: 9999999\r\n
Content-Type: application/octet-stream\r\n\r\n

// 然后持续发送音频数据
```

- 如果用if-else会怎样？

- ```
// 反模式：难以维护
fun startAudioCall() {
    if (deviceType == "YUSHI") {
        // 100行握手代码
    } else if (deviceType == "KEDA") {
        // 50行协议头代码
    } else if (deviceType == "WDZ_NVR") {
        // 30行HTTP模拟代码
    }
    // 新增设备时要改这里，违反开闭原则
}
```

- 解决思路：策略模式 + 工厂模式，解耦。这个过会看我具体的架构设计

挑战2：音频处理的线程协调

- 传统单线程做法：采集 → 编码 → 发送（串行执行）

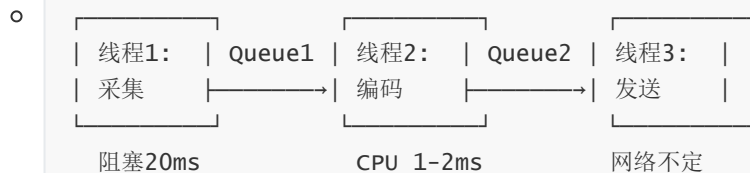
- 采集 → 编码 → 发送（串行执行）

问题分析：

- └─ `AudioRecord.read()` 阻塞 20-40ms
- └─ `G.711`编码耗时 1-2ms
- └─ `Socket.send()` 可能阻塞（网络慢）
- └─ 总延迟累加，容易掉帧

- 每个阻塞一点，总延迟累加，掉帧率很高。

- 解决思路：流水线并行处理，让每个阶段完全独立开来运行



优势:

- 每个阶段独立运行
- 队列缓冲, 削峰填谷
- 某阶段慢不会拖累其他阶段

挑战3: 内存和GC压力

- 传统方法: 每次采样, new一个对象, 将数据从audioRecord的native层读取到java堆中, 然后进行编码等处理, 并且发送

○

```

// 采样率8000Hz, 16bit, 单声道
// 每次读取640字节 (40ms数据)
// 每秒产生: 1000ms / 40ms = 25个buffer对象

// ✕ 传统做法
fun capture() {
    while (true) {
        val buffer = ByteArray(640) // 每次new, 每秒25个对象
        audioRecord.read(buffer, 0, 640)
        encode(buffer) // 又产生编码后的对象
    }
}
// 每秒至少50个对象, 频繁触发GC!
  
```

- 解决思路: 对象池复用

○

```

val pool = ByteBufferPool(bufferSize = 640, maxPoolSize = 10)

fun capture() {
    while (true) {
        val buffer = pool.acquire() // 从池中获取
        // ... 使用buffer
        pool.release(buffer) // 归还到池中
    }
}

// 效果: 只创建10个对象, 复用率95%以上, GC大幅降低
  
```

2.3 技术选型

约束条件

约束	原因	影响
☒ 必须用G.711编码	服务端只支持G.711	不能用MediaCodec的AAC/Opus优化
☒ 必须用TCP Socket	服务端不支持WebSocket	不能用现成的WebSocket库

选型决策：

- 本质是个双生产者-消费者模型。涉及到：生产，消费生产，消费这个过程。

技术点	方案对比	最终选择	理由
采样方案	系统自带的audioRecord（无方案对比）	audioRecord	系统提供的能力
编码方案	MediaCodec (AAC/Opus) vs 纯java，调用cpu G.711	☒ G.711	服务端约束
传输方案	WebSocket vs TCP Socket	☒ TCP Socket	服务端约束
线程模型	单线程 vs 多线程流水线	☒ 三线程流水线	性能和解耦
内存方案	每次new vs 对象池	☒ ByteBufferPool	降低GC
不同设备适配	if-else vs 策略模式	☒ 策略模式	可扩展性
组件间的协调	单一类 vs Manager协调	☒ Manager模式	职责清晰

为什么服务端只支持G.711？

- 硬件摄像头普遍使用G.711（行业标准），解码实现简单，无需复杂的DSP

为什么不用WebSocket？

- 设备固件老旧，只支持原始TCP
- 厂商私有协议，不遵循标准
- 升级设备固件成本高（大量存量设备）

2.4 设计模式

本质是SOLID原则的体现

- S - 单一职责原则 (Single Responsibility)

- ☒ IAudioSource → 只负责音频采集（AudioRecord封装）
- ☒ IEncoder → 只负责编码（PCM → G.711）
- ☒ IAudioChannel → 只负责网络传输（TCP Socket封装）
- ☒ IDeviceStrategy → 只负责设备适配（握手协议）
- ☒ IAudioProcessor → 只负责流水线协调（三线程管理）
- ☒ VoiceIntercomManager → 只负责组件组装（不实现具体功能）

// 每个类都有明确的职责，不越界

- O - 开闭原则 (Open-Closed)

- 对拓展开放：新增设备类型，无需修改现有代码；对修改关闭：核心类无需改动，只用在策略工厂中添加一个case即可

- ```
// ☒ 对扩展开放
class NewDeviceStrategy : IDeviceStrategy {
 // 新增设备类型，无需修改现有代码
```

```

}

// ☒ 对修改关闭
// VoiceIntercomManager、AudioProcessorImpl 等核心类无需改动
// 只需在DeviceStrategyFactory中增加一个case

object DeviceStrategyFactory {
 fun createStrategy(deviceType: DeviceType): IDeviceStrategy {
 return when (deviceType) {
 DeviceType.WDZ_NVR -> WdznvrStrategy()
 DeviceType.YUSHI -> YuShiStrategy()
 DeviceType.KEDA -> KedaStrategy()
 DeviceType.NEW_DEVICE -> NewDeviceStrategy() // 只改这里
 }
 }
}

```

- L - 里氏替换原则 (Liskov Substitution)

- 通过接口替换不同的策略

```

// 任何IDeviceStrategy的实现都可以替换使用
fun test(strategy: IDeviceStrategy) {
 strategy.configureChannel(channel, config, listener)
 val encoder = strategy.createEncoder()
 val rate = strategy.getSampleRate()
}

// 可以传入任何实现
test(YuShiStrategy()) // ☒ 工作正常
test(KedaStrategy()) // ☒ 工作正常
test(WdznvrStrategy()) // ☒ 工作正常

// Manager的代码无需改变

```

- I - 接口隔离原则 (Interface Segregation)

- ```
// ☒ 不好的设计：大而全的接口
interface IChannel {
    fun connect()
    fun disconnect()
    fun send()
    fun receive()
    fun setListener()
}

// 问题：使用者可能只需要send功能，却要实现全部方法

// ☒ 好的设计：职责分离
interface IConnectable {
    fun connect(host: String, port: Int, listener:
OnConnectionListener?)
    fun disconnect()
    fun isConnected(): Boolean
}

interface ISendable {
    fun send(data: ByteArray)
    fun sendString(message: String)
```



```

}

interface IReceivable {
    fun setOnDataReceivedListener(listener: OnDataReceivedListener?)
}

// 组合接口
interface IAudioChannel : IConnectable, ISendable, IReceivable

// 组件只依赖需要的接口
class AudioSender(private val sendable: ISendable) {
    // 只需要发送功能，不关心连接和接收
}

```

- D - 依赖倒置原则 (Dependency Inversion)

- ☒ 高层模块依赖抽象接口，不依赖具体实现

```

class VoiceIntercomManager {
    // 依赖抽象接口
    private var audioSource: IAudioSource? // 接口
    private var encoder: IEncoder? // 接口
    private var channel: IAudioChannel? // 接口
    private var deviceStrategy: IDeviceStrategy? // 接口

    fun start(config: VoiceIntercomConfig) {
        // 具体实现在运行时注入
        audioSource = AudioCapturerImpl() // 具体类
        encoder = G711Encoder(Type.PCMA) // 具体类
        channel = TcpAudioChannel() // 具体类

        // 易于测试：可以注入Mock对象
        audioSource = MockAudioSource() // 单元测试时
    }
}

```

2.5 性能指标

开发结果性能判定指标

- 我们怎么知道开发是成功的？我们制定了明确的量化指标，从3个维度验证
 - 可靠性维度：丢包率（这个是指系统采样的帧是否全部成功发出去了，这个链路中丢了多少？）
 - 性能维度：
 - 启动延迟
 - 发送链路延迟
 - 内存分配率
 - GC频率
 - 稳定性维度
 - 队列深度（是否队列被占满，是否劣化？产生挤压？）
 - cpu使用率？（cpu占用率是否过高？抖动是否过高？）
 - 新策略添加是否方便（举例：新的设备，其拥有新的握手和加密方式，同时支持新的编码格式，也支持新的传输格式，这就是对整体架构的重新处理了。这就考虑我们的整体性能了。）

下面我们看看指标详解

- 丢包率

- $\text{丢包率} = \text{丢弃帧数} / \text{总帧数} \times 100\%$

// 统计位置

AudioProcessorImpl.kt:

- pcmDropped: PCM队列满时丢弃
- encodedDropped: Encoded队列满时丢弃
- totalDropped: 设备策略canSendAudio()返回false时丢弃

判断标准:

优秀: <0.5% (几乎无损)

良好: 0.5% - 1% (偶尔丢帧)

可接受: 1% - 3% (有轻微卡顿)

不合格: >3% (明显卡顿)

本项目实测: 0.15%

- 总发送: 1998帧
- 总丢弃: 3帧 (仅启动初期, 字幕握手阶段)
- BridgeReady后: 0丢帧

- 连接延时 (也就是按下按钮到初始化完成的时间)

- 启动延迟 = 从调用start()到AudioProcessor就绪的时间

// PerformanceMonitor统计

- TotalStart: 总启动时间
- Connection: TCP连接时间
- AudioInit: AudioRecord初始化时间

判断标准:

优秀: <200ms (用户无感知)

良好: 200ms - 500ms (可接受)

不合格: >500ms (明显延迟)

本项目实测: 126ms ☒

- TCP连接: 96ms (76.2%)
- 音频初始化: 29ms (23.0%)

- 内存分配率 (应该说是内存命中率, 是命中的次数/申请的次数)

- 内存分配率 = 每秒新分配的字节数

// 优化手段

ByteBufferPool命中率 = (acquire次数 - create次数) / acquire次数

命中率越高 → 新对象创建越少 → 内存分配率越低

判断标准:

优秀: 命中率 >95%

良好: 命中率 80% - 95%

需优化: 命中率 <80%

本项目实测: 99.75% ☒

- Acquire次数: 1998次
- Create次数: 5次
- 内存分配: 3.2KB (vs 无池化的1.22MB)
- 减少: 99.74%

- GC率 (Garbage Collection Frequency)

- GC率 = 单位时间内的GC次数

关注指标:

1. GC频率: 每秒多少次
2. GC时长: 每次多久 (特别是暂停时长)
3. GC类型: Young GC vs Full GC

- 队列挤压

- 队列利用率 = 当前队列深度 / 队列容量 × 100%

// 监控位置

- pcmQueue.size() / QUEUE_CAPACITY
- encodedQueue.size() / QUEUE_CAPACITY

判断标准:

健康: <30% (轻负载, 有余量)

正常: 30% - 50% (中等负载)

警告: 50% - 80% (高负载, 有挤压风险)

危险: >80% (即将丢帧)

意义:

队列深度低: 说明生产者-消费者速率匹配良好

队列深度高: 说明消费者速度跟不上, 有瓶颈

- pcmQueue平均深度: 2.3帧 / 20帧 = 11.5% ☒
 - encodedQueue平均深度: 1.8帧 / 20帧 = 9.0% ☒
- 结论: 三线程并行度高, 无瓶颈

- 总结

- "我们制定了明确的量化指标, 从3个维度验证:

1. 可靠性维度:

- 丢包率0.15% (<1%标准) ☒
- 80秒测试, 握手后0丢帧 ☒

2. 性能维度:

- 启动延迟126ms (<200ms标准) ☒
- 内存分配减少99.74% ☒
- GC频率减少93.75% ☒

3. 稳定性维度:

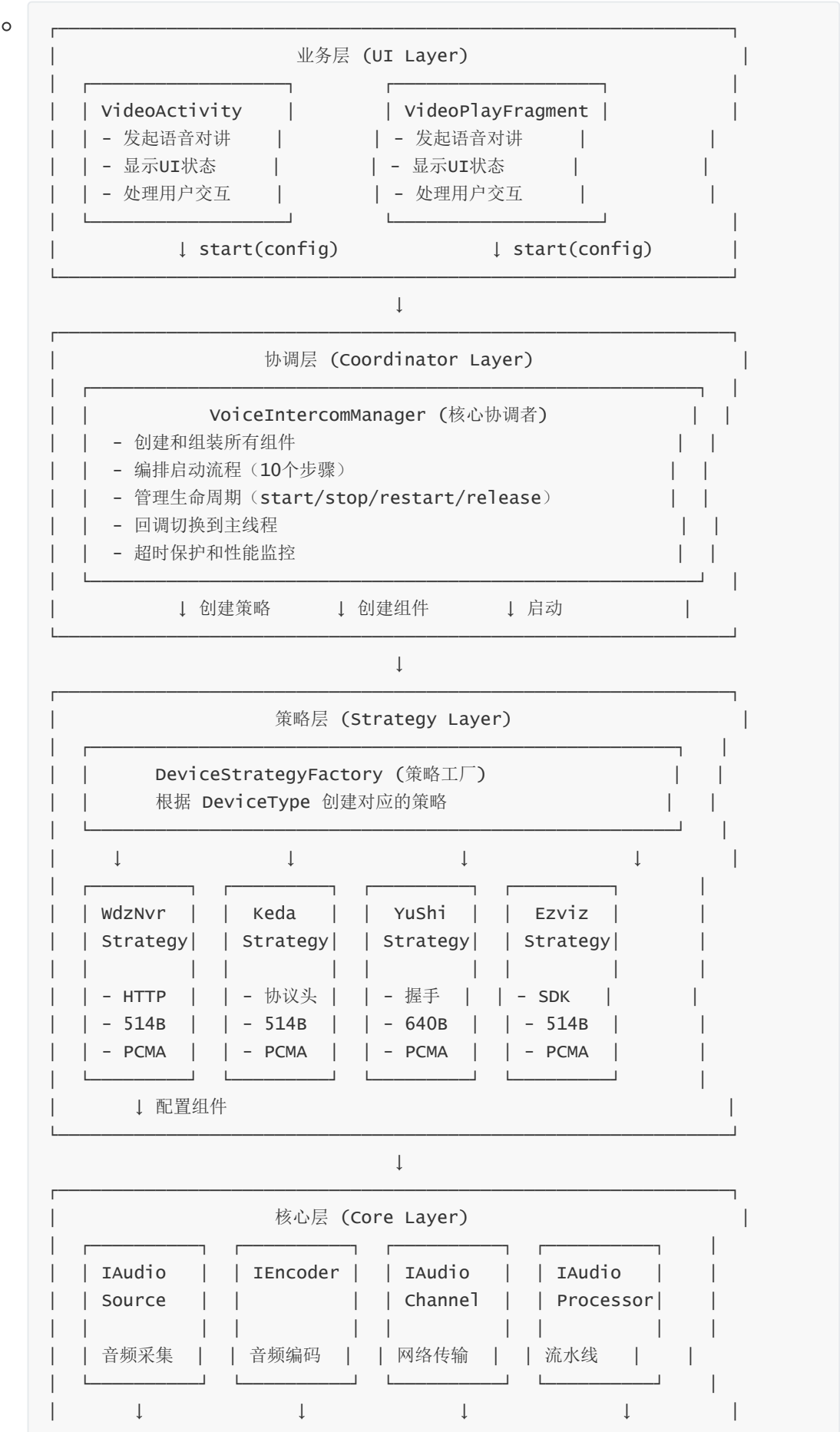
- 队列深度11.5% (无挤压) ☒
- 帧率精度99.9% (生产消费速率匹配) ☒
- CPU使用率<10% ☒

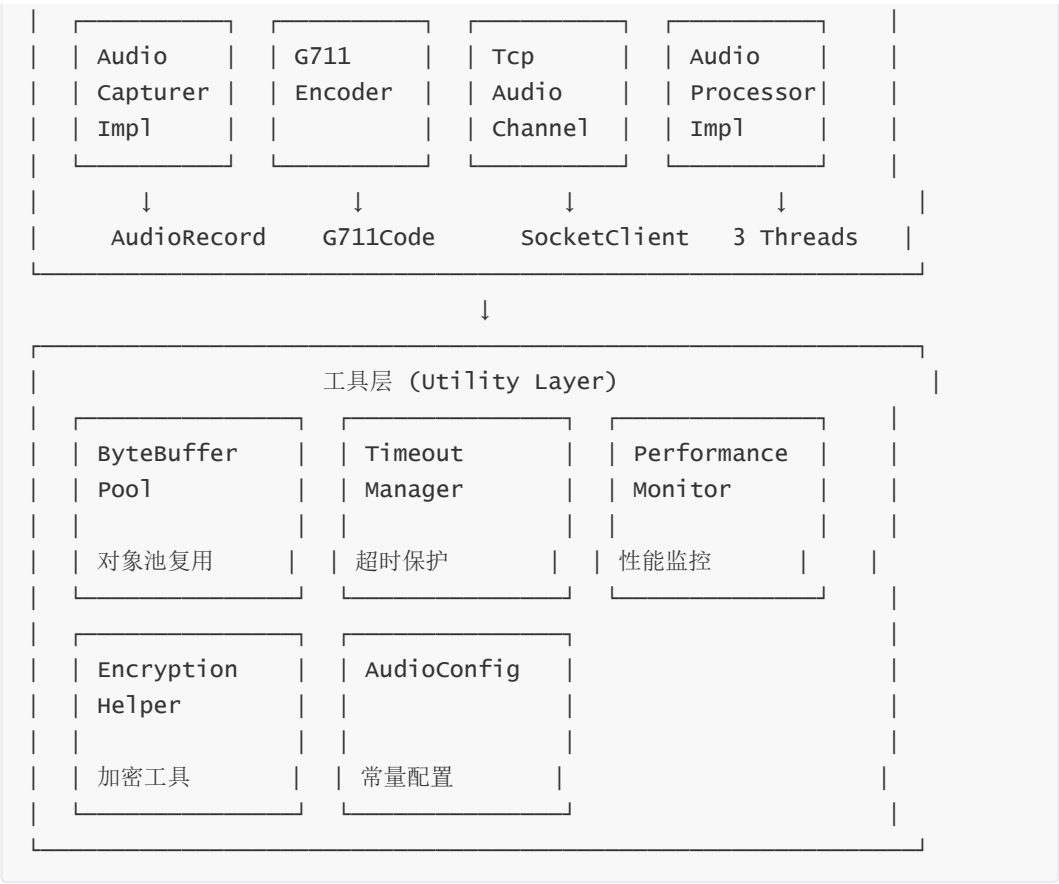
这些数据都是通过PerformanceMonitor埋点和Android Profiler实测得出的, 证明重构达到了预期目标。"

03.整体架构设计

3.1 五层架构全景图

- 架构图解





- 架构分层职责说明，如下：

。 层级	职责	核心类	特点
业务层	UI交互、状态展示	VideoActivity	调用Manager，处理回调
协调层	组件组装、流程编排	VoiceIntercomManager	只负责协调，提供生命周期
策略层	设备差异封装	IDeviceStrategy及实现类	策略模式，易扩展
核心层	音频采集/编码/传输	AudioSource、Encoder、Channel、Processor	职责单一，可独立测试
工具层	通用工具和配置	Pool、TimeoutManager、Monitor	横向支持，无业务逻辑

3.2 数据流转路径

- 完整数据流（采集 → 编码 → 发送）



AudioRecord (Android系统)
↓ 每40ms产生640字节PCM数据
AudioCapturerImpl.read(buffer: ByteBuffer)
↓ 阻塞读取, 直到有数据
ByteBuffer (从ByteBufferPool获取)
↓ 计算音量 (300ms更新一次)
pcmQueue.offer(buffer) ← 非阻塞, 队列满则丢弃
↓

Queue1: pcmQueue (容量20)
存储原始PCM数据, 解耦采集和编码

↓

阶段2: 音频编码 (EncoderThread)

pcmQueue.poll(100ms) ← 阻塞等待, 超时返回null
↓ 有数据则继续
ByteBuffer → ByteArray (转换)
↓ 640字节
G711Encoder.encode(pcmData: ByteArray)
↓ 2:1压缩
ByteArray (320字节, G.711数据)
↓ 回收ByteBuffer到池
encodedQueue.offer(encodedData) ← 非阻塞
↓

Queue2: encodedQueue (容量20)
存储编码后的数据, 解耦编码和发送

↓

阶段3: 网络发送 (SenderThread)

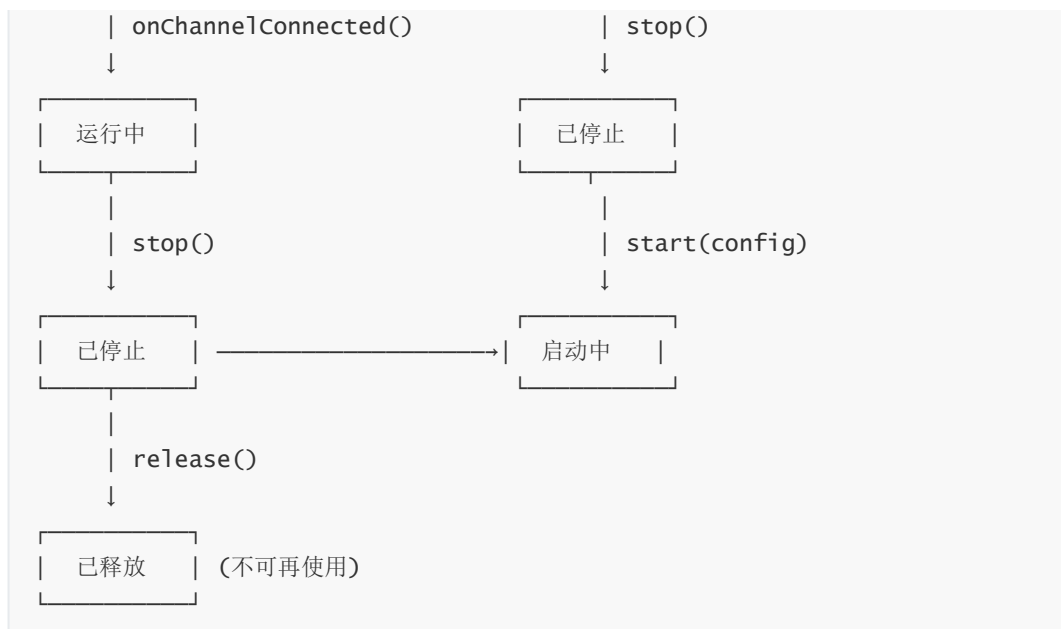
encodedQueue.poll(100ms) ← 阻塞等待
↓ 有数据则继续
deviceStrategy.canSendAudio() ← 检查是否可以发送
↓ true则发送, false则丢弃
TcpAudioChannel.send(encodedData: ByteArray)
↓ 320字节
SocketClient.send()
↓ 通过TCP发送到服务端
[服务端摄像头] 播放音频

3.3 生命周期管理

- Manager的生命周期状态机

○





- start() 方法详细流程

```

○ fun start(config: VoiceIntercomConfig) {
    // ===== 步骤1: 状态检查 =====
    if (isRunning) {
        Log.w(TAG, "Already running!")
        return
    }

    // ===== 步骤2: 创建设备策略 =====
    deviceStrategy =
DeviceStrategyFactory.createStrategy(config.deviceType)
    // 根据deviceType创建对应的策略: YuShi/Keda/wdzNvr/Ezviz

    // ===== 步骤3: 创建核心组件 =====
    audioSource = AudioCapturerImpl()
    encoder = deviceStrategy?.createEncoder() // 策略决定编码器类型
    channel = TcpAudioChannel()
    audioProcessor = AudioProcessorImpl()

    // ===== 步骤4: 配置AudioSource =====
    deviceStrategy?.configureAudioSource(audioSource)
    // 例如: 宇视设置640字节, 其他设备514字节

    // ===== 步骤5: 配置Channel =====
    val deviceConfig = DeviceConfig.fromVoiceIntercomConfig(config)
    channel?.configure(deviceConfig)

    // ===== 步骤6: 初始化AudioProcessor =====
    audioProcessor?.init(audioSource, encoder, channel, deviceStrategy)
    // 注入所有依赖

    // ===== 步骤7: 配置Channel连接 (异步) =====
    deviceStrategy?.configureChannel(channel, deviceConfig, object :
onConnectionListener {
        override fun onConnected() {
            // 连接成功, 进入步骤8
            onChannelConnected(config)
        }
    })
}

```

```

        override fun onDisconnected() { ... }
        override fun onError(e: Exception) { ... }
    })
    // 策略内部会调用 channel.connect() 并设置握手逻辑

    // ===== 步骤8: 连接成功回调 =====
    // 见 onChannelConnected() 方法
}

private fun onChannelConnected(config: VoiceIntercomConfig) {
    // 防止重复调用
    if (isRunning) return

    // 取消总体超时
    timeoutManager.cancelTimeout()

    isRunning = true
    callback?.onConnected() // 通知业务层

    // 初始化AudioRecord
    val sampleRate = deviceStrategy?.getSampleRate() ?:
config.sampleRate
    val success = audioSource?.init(sampleRate)
    if (!success) {
        callback?.onError(Exception("AudioRecord init failed"))
        stop()
        return
    }

    // 启动流水线
    audioProcessor?.start()

    // 输出性能统计
    performanceMonitor.recordEnd(NODE_TOTAL_START)
    performanceMonitor.logStatistics()
}

```

- stop方法详解

- ```

fun stop() {
 if (!isRunning) return

 isRunning = false

 // 步骤1: 停止音频处理器（会停止3个线程）
 audioProcessor?.stop()

 // 步骤2: 停止音频源
 audioSource?.stop() // 释放AudioRecord

 // 步骤3: 断开通道
 channel?.disconnect() // 关闭Socket

 // 步骤4: 清理资源
 cleanup() // 将所有组件置为null
}

```



## 3.4 架构的可扩展性

- 可扩展性如下

- ```
// 场景1: 新增设备类型
// 只需3步:
1. 添加枚举: DeviceType.NEW_DEVICE
2. 实现策略: class NewDeviceStrategy : IDeviceStrategy { ... }
3. 注册工厂: DeviceStrategyFactory { ... case NEW_DEVICE ->
NewDeviceStrategy() }

// 场景2: 替换编码器 (假设支持其他编码)
// 只需2步:
1. 实现接口: class OpusEncoder : IEncoder { ... }
2. 策略返回: override fun createEncoder() = OpusEncoder()

// 场景3: 替换传输层 (假设支持UDP)
// 只需2步:
1. 实现接口: class UdpAudioChannel : IAudioChannel { ... }
2. Manager创建: channel = UdpAudioChannel()

// 核心代码无需改动!
```

04.协调层 - VoiceIntercomManager

4.1 Manager的职责和定位

- 他是一个纯粹的协调者，他要知道：我们需要哪些人，如何把它们组建起来，完成任务的工作流程是什么样的，这个过程中如何协调各方，他不做具体的工作

- ```
/**
 * VoiceIntercomManager - 纯协调者
 *
 * 它就像一个"项目经理":
 * - ☒ 知道需要哪些人 (组件)
 * - ☒ 知道如何组建团队 (依赖注入)
 * - ☒ 知道工作流程 (启动步骤)
 * - ☒ 协调各方合作 (回调转发)
 * - ☒ 监测工作过程中的成员状态和任务完成情况 (性能监控)
 *
 * 但它不做具体工作:
 * - X 不采集音频
 * - X 不编码数据
 * - X 不发送网络包
 */
class VoiceIntercomManager private constructor(
 private val callback: VoiceIntercomCallback?
) {
 // 组件引用 (全部是接口)
 private var audioSource: IAudioSource? = null
 private var encoder: IEncoder? = null
 private var channel: IAudioChannel? = null
 private var deviceStrategy: IDeviceStrategy? = null
 private var audioProcessor: IAudioProcessor? = null

 // 辅助工具
```

```

private val mainHandler = Handler(Looper.getMainLooper())
private val timeoutManager = TimeoutManager()
private val performanceMonitor = PerformanceMonitor()

// 状态标志
@Volatile private var isRunning = false
}

```

## 4.2 核心方法：start() 深度解析

- 完整代码

```

○ fun start(config: VoiceIntercomConfig) {
 // ===== 步骤1: 状态检查 =====
 if (isRunning) {
 Log.w(TAG, "VoiceIntercom already running!")
 return
 }

 Log.d(TAG, "Starting voice intercom:
deviceType=${config.deviceType}, host=${config.host}:${config.port}")

 // ===== 步骤2: 启动性能监控 =====
 performanceMonitor.recordStart(PerformanceMonitor.NODE_TOTAL_START)
 // 作用: 记录总启动时间, 用于性能分析

 // ===== 步骤3: 启动总体超时保护 =====
 timeoutManager.startTimeout(AudioConfig.TOTAL_START_TIMEOUT_MS,
"VoiceIntercomStart") {
 // 20秒后仍未启动成功, 强制失败
 Log.e(TAG, "Voice intercom start timeout after
${AudioConfig.TOTAL_START_TIMEOUT_MS}ms")
 mainHandler.post {
 callback?.onError(Exception("Start timeout"))
 }
 stop()
 }
 // 为什么需要? 防止某个环节卡住导致永久等待

 try {
 // ===== 步骤4: 创建设备策略 =====
 deviceStrategy =
DeviceStrategyFactory.createStrategy(config.deviceType)
 Log.d(TAG, "Device strategy created:
${deviceStrategy?.javaClass?.simpleName}")

 // 工厂根据deviceType返回对应的策略:
 // - DeviceType.YUSHI → YuShiStrategy()
 // - DeviceType.KEDA → KedaStrategy()
 // - DeviceType.WDZ_NVR → WdzNvrStrategy()
 // - DeviceType.EZVIZ → EzvizStrategy()

 // ===== 步骤5: 创建核心组件 =====
 audioSource = AudioCapturerImpl()
 encoder = deviceStrategy?.createEncoder() // 策略决定编码器
 channel = TcpAudioChannel()
 audioProcessor = AudioProcessorImpl()
 }
}

```

```

 Log.d(TAG, "Core components created: source=${audioSource},
encoder=${encoder}, channel=${channel}")

 // ===== 步骤6: 配置AudioSource
 =====
 deviceStrategy?.configureAudioSource(audioSource)
 // 策略会设置buffer大小:
 // - 字视: 640字节 (固定)
 // - 其他: 514字节 (标准)

 // ===== 步骤7: 配置Channel =====
 val deviceConfig = DeviceConfig.fromVoiceIntercomConfig(config)
 channel?.configure(deviceConfig)
 // 设置host、port、timeout等参数

 // ===== 步骤8: 初始化AudioProcessor
 =====
 val source = requireNotNull(audioSource) { "AudioSource not
created" }
 val enc = requireNotNull(encoder) { "Encoder not created" }
 val chan = requireNotNull(channel) { "Channel not created" }
 val strategy = requireNotNull(deviceStrategy) { "DeviceStrategy
not created" }
 val processor = requireNotNull(audioProcessor) {
"AudioProcessor not created" }

 processor.init(source, enc, chan, strategy)
 // 注入所有依赖, 但不启动

 // 设置Processor监听器
 processor.setProcessorListener(object :
IAudioProcessor.OnProcessorListener {
 override fun onError(e: Exception) {
 Log.e(TAG, "Processor error", e)
 mainHandler.post {
 callback?.onError(e)
 }
 }

 override fun onVolumeChanged(volume: Int) {
 mainHandler.post {
 callback?.onAudioVolume(volume)
 }
 }
 })

 // ===== 步骤9: 配置Channel连接 (关键!)
 =====
 // 这是异步操作, 需要等待连接成功

 // 针对字视设备: 需要将握手消息回传给VideoActivity
 (strategy as? YuShiStrategy)?.setMessageListener { message ->
 mainHandler.post {
 callback?.onResponse(message)
 }
 }
 }

```

```

// 开始连接监控

performanceMonitor.recordStart(PerformanceMonitor.NODE_CONNECTION)

// 策略会调用 channel.connect() 并设置握手逻辑
strategy.configureChannel(chan, deviceConfig, object :
IConnectable.OnConnectionListener {
 override fun onConnected() {
 // 连接成功! 进入步骤10

performanceMonitor.recordEnd(PerformanceMonitor.NODE_CONNECTION)

 // 策略连接成功, 启动音频采集和处理
 onChannelConnected(config)
 }

 override fun onDisconnected() {
 isRunning = false
 mainHandler.post {
 callback?.onDisconnected()
 }
 }

 override fun onError(e: Exception) {
 Log.e(TAG, "Channel error", e)
 mainHandler.post {
 callback?.onError(e)
 }
 cleanup()
 }
})

// ===== 步骤10: 等待异步连接 =====
// 此时start()方法返回, 等待onChannelConnected()回调

} catch (e: Exception) {
 Log.e(TAG, "Failed to start voice intercom", e)
 mainHandler.post {
 callback?.onError(e)
 }
 cleanup()
}
}

```

- onChannelConnected() 详解

```

o /**
 * 连接成功回调 - 启动音频流水线
 *
 * 为什么单独拆出这个方法?
 * 1. start()是同步方法, 连接是异步的
 * 2. 宇视设备需要握手, 连接成功*可以发送音频
 * 3. 逻辑清晰, 易于测试
 */
private fun onChannelConnected(config: VoiceIntercomConfig) {
 // 防止重复调用 (宇视设备可能触发两次)
 if (isRunning) {

```

```

 Log.w(TAG, "Already running, ignore duplicate connection
callback")
 return
 }

 Log.d(TAG, "Channel connected successfully")

 // 取消总体超时（连接成功）
 timeoutManager.cancelTimeout()

 isRunning = true

 // 通知业务层：连接成功
 mainHandler.post {
 callback?.onConnected()
 }

 // ===== 初始化AudioRecord =====
 // 为什么在这里初始化？
 // 1. AudioRecord初始化耗时（可能200-500ms）
 // 2. 等连接成功后再初始化，避免浪费资源
 // 3. 如果连接失败，不会占用麦克风

 val sampleRate = deviceStrategy?.getSampleRate() ?:
config.sampleRate
 Log.d(TAG, "Initializing audio source: sampleRate=$sampleRate Hz")

 // 监控AudioRecord初始化耗时
 performanceMonitor.recordStart(PerformanceMonitor.NODE_AUDIO_INIT)
 val success = audioSource?.init(sampleRate)
 performanceMonitor.recordEnd(PerformanceMonitor.NODE_AUDIO_INIT)

 if (!success) {
 Log.e(TAG, "Failed to initialize audio source")
 mainHandler.post {
 callback?.onError(Exception("Failed to initialize audio
source"))
 }
 stop()
 return
 }

 // ===== 启动Processor流水线 =====
 Log.d(TAG, "Starting audio processor...")

 performanceMonitor.recordStart(PerformanceMonitor.NODE_PROCESSOR_START)
 audioProcessor?.start()

 performanceMonitor.recordEnd(PerformanceMonitor.NODE_PROCESSOR_START)

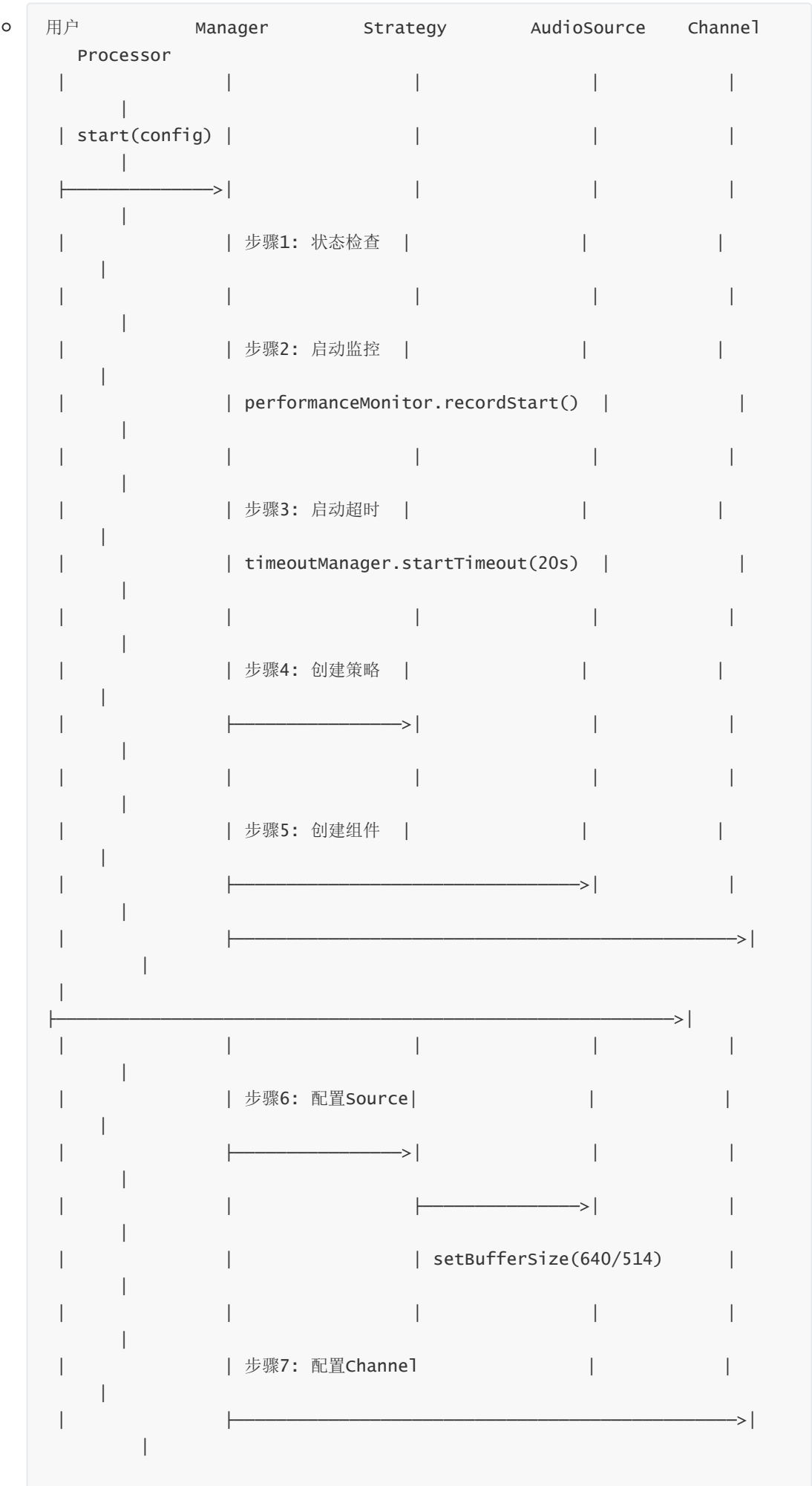
 // ===== 完成总体启动 =====
 performanceMonitor.recordEnd(PerformanceMonitor.NODE_TOTAL_START)

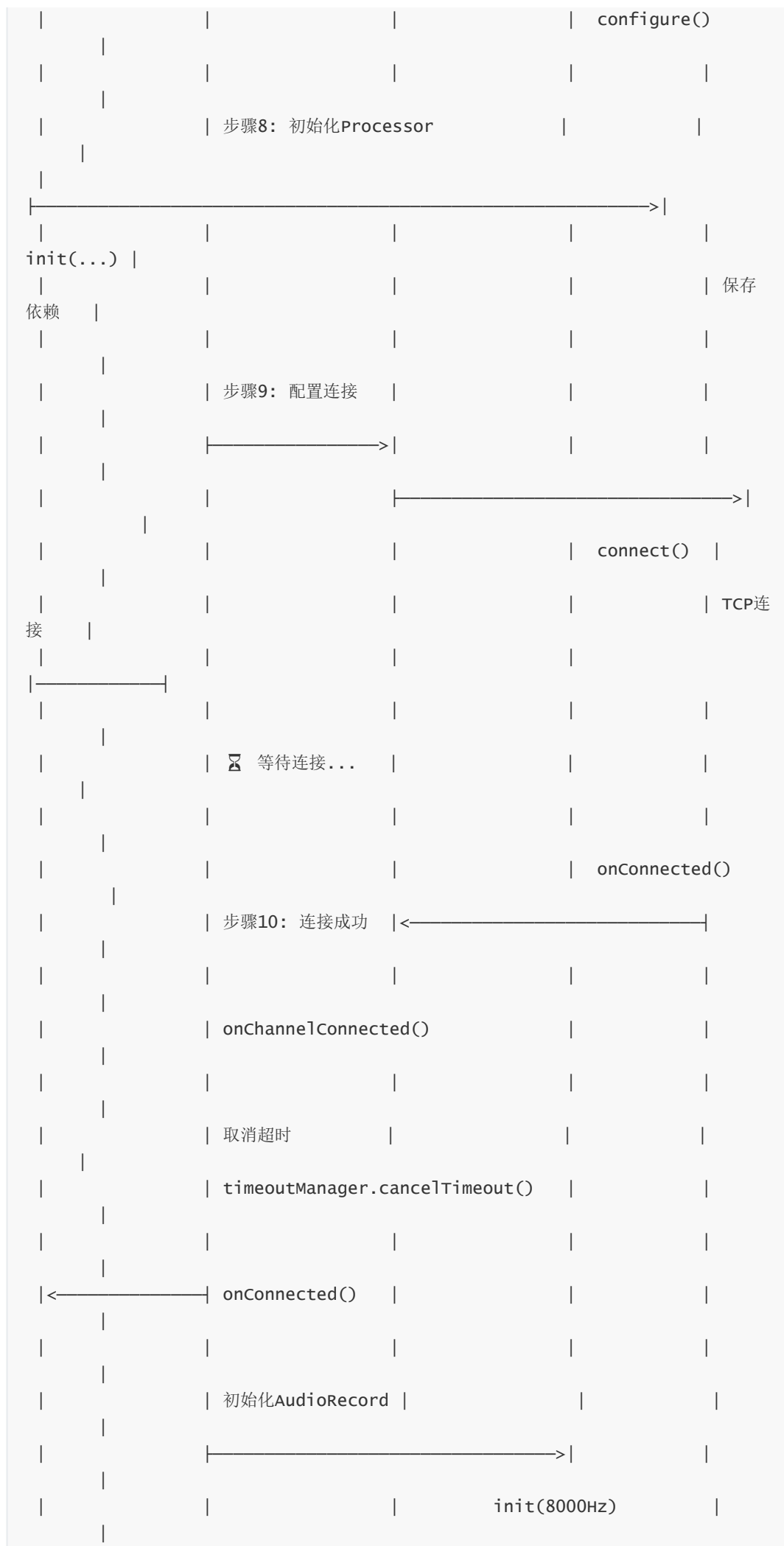
 // 输出性能统计
 performanceMonitor.logStatistics()

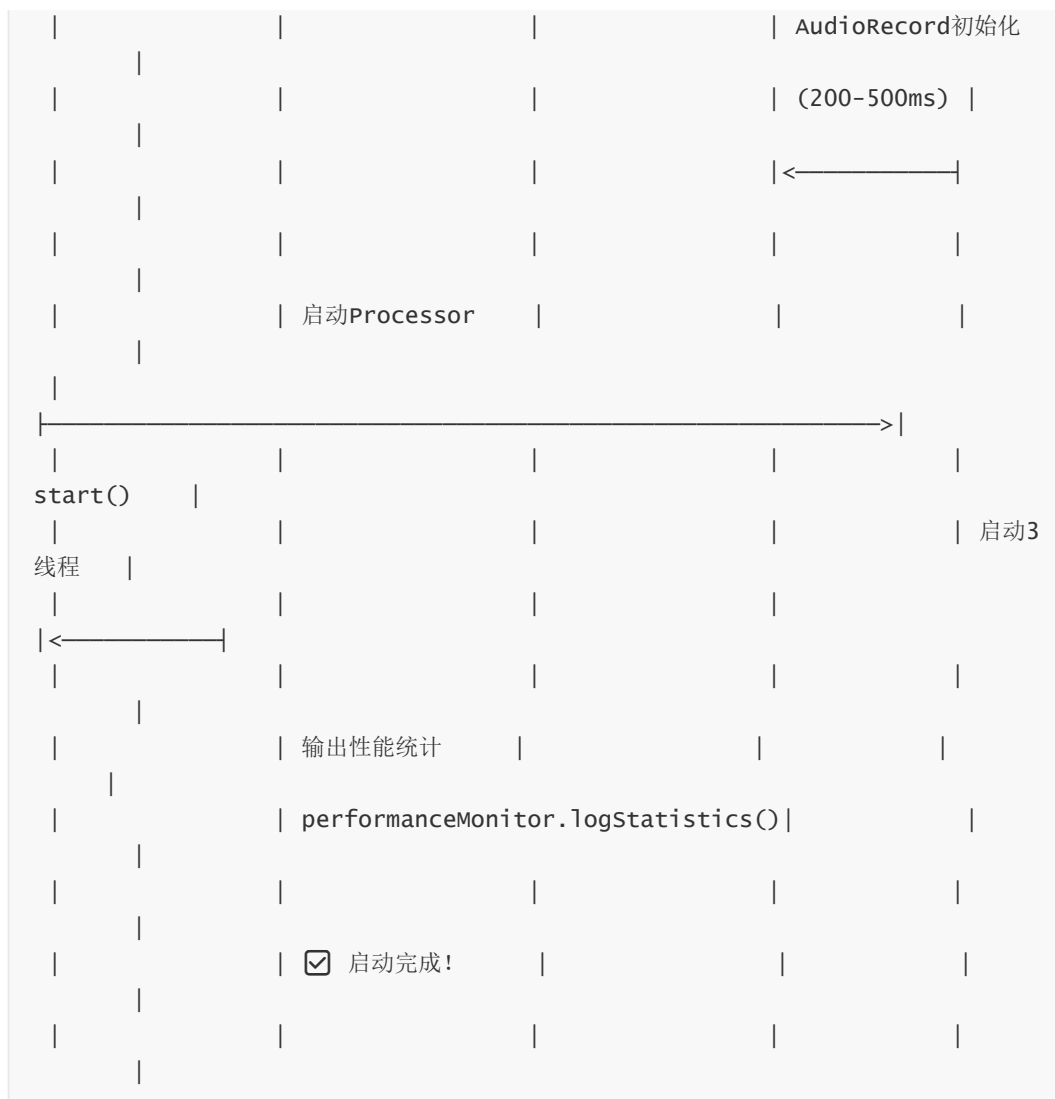
 Log.d(TAG, "Voice intercom fully started!")
}

```

• 时序图 (详细版)







## 4.3 关键设计解析

- 为什么start()不直接初始化AudioRecord?
  - AudioRecord初始化涉及到底层系统组件的控制，消耗资源和时间，因此不应该在连接成功之前就进行相关的初始化。
- 为什么需要防重复调用?
  - 宇视设备握手流程可能触发两次连接回调：TCP连接成功；BridgeReady消息到达

```

private fun onChannelConnected(config: VoiceIntercomConfig) {
 if (isRunning) {
 Log.w(TAG, "Already running, ignore duplicate connection callback")
 return
 }
 // ...
}

```

原因：

宇视设备握手流程可能触发两次连接回调：

1. TCP连接成功
2. BridgeReady消息到达

防重复可以避免：

- └─ AudioRecord重复初始化（报错）
- └─ Processor重复启动（资源泄漏）



- 如何使用Builder模式?

```
○ class VoiceIntercomManager private constructor(
 private val callback: VoiceIntercomCallback?
) {
 // 私有构造函数, 只能通过Builder创建

 companion object {
 private const val TAG = "VoiceIntercomManager"
 }

 // ... Manager实现

 /**
 * Builder模式
 */
 class Builder {
 // Builder的参数
 private var callback: VoiceIntercomCallback? = null

 /**
 * 设置回调
 */
 fun setCallback(callback: VoiceIntercomCallback): Builder {
 this.callback = callback
 return this // 返回自己, 支持链式调用
 }

 /**
 * 构建Manager实例
 */
 fun build(): VoiceIntercomManager {
 // 参数校验 (可选)
 // if (callback == null) {
 // throw IllegalArgumentException("Callback is
required")
 // }

 // 创建Manager实例
 return VoiceIntercomManager(callback)
 }
 }
}
```

## 05.策略层 - 设备适配策略

### 5.1 策略接口设计 IDeviceStrategy

- 接口定义与职责

```
○ /**
 * 设备策略接口 - 设备特定的适配逻辑
 *
 * 职责:
```

- \* 1. 配置AudioSource（如字幕需要640字节buffer）
- \* 2. 创建Encoder（返回对应的编码器实例）
- \* 3. 配置Channel握手（设备特有的连接协议）
- \* 4. 提供采样率（不同设备可能不同）
- \* 5. 控制发送时机（如字幕需要等握手完成）

```

*/
interface IDeviceStrategy {

 /**
 * 配置音频源
 *
 * @param audioSource 音频源实例
 *
 * 作用：设置设备特定的配置
 * - 字幕：640字节（固定，协议要求）
 * - 其他：514字节（标准大小）
 */
 fun configureAudioSource(audioSource: IAudioSource?)

 /**
 * 创建编码器
 *
 * @return 编码器实例
 *
 * 作用：根据设备支持的编码格式创建对应编码器
 * - 大部分设备：PCMA (A-law)
 * - 少数设备：PCMU (μ-law)
 */
 fun createEncoder(): IEncoder?

 /**
 * 配置通道（最核心的方法）
 *
 * @param channel 音频通道实例
 * @param config 设备配置
 * @param connectionListener 连接监听器
 *
 * 作用：设置设备特有的连接流程
 * - 字幕：复杂的4步握手协议
 * - 科达：发送协议头
 * - 自研NVR：发送HTTP header
 *
 * 注意：策略负责调用 channel.connect() 并设置握手逻辑
 */
 fun configureChannel(
 channel: IAudioChannel?,
 config: DeviceConfig?,
 connectionListener: IConnectable.OnConnectionListener?
)

 /**
 * 获取采样率
 *
 * @return 采样率（Hz）
 *
 * 作用：提供设备支持的采样率
 * - 大部分设备：8000Hz（电话音质）
 * - 高清设备：16000Hz（预留，暂未使用）
 */
}

```

```

 */
 fun getSampleRate(): Int

 /**
 * 是否可以发送音频
 *
 * @return true 可以发送, false 需要等待
 *
 * 作用: 控制发送时机
 * - 大部分设备: 连接成功即可发送, 返回 true
 * - 宇视设备: 需要等握手完成 (BridgeReady), 初始返回 false
 *
 * 默认实现: 返回 true (大部分设备适用)
 */
 fun canSendAudio(): Boolean = true
}

```

- 接口设计方法的权衡

| 方法                                | 为什么这样设计?         | 如果不这样设计会怎样?            |
|-----------------------------------|------------------|------------------------|
| <code>configureAudioSource</code> | 允许策略定制 buffer 大小 | ✗ 宇视设备640字节的需求无法满足     |
| <code>createEncoder</code>        | 策略决定编码器类型        | ✗ 如果设备需要不同编码格式, 无法扩展   |
| <code>configureChannel</code>     | 策略完全控制握手流程       | ✗ 宇视的复杂握手无法实现          |
| <code>getSampleRate</code>        | 策略提供采样率          | ✗ 如果设备需要不同采样率, 写死会出问题  |
| <code>canSendAudio</code>         | 策略控制发送时机         | ✗ 宇视设备会在握手前就发送数据, 导致失败 |

- 其实目前项目中的设备基本都是握手流程的不同以及buffer的大小不同, 但是如果后续有更先进的设备, 可以支持WebSocket和MediaCodec技术, 那么可以通过这些来更换其技术实现

## 5.2 宇视设备策略 YuShiStrategy

- 宇视设备的特殊性

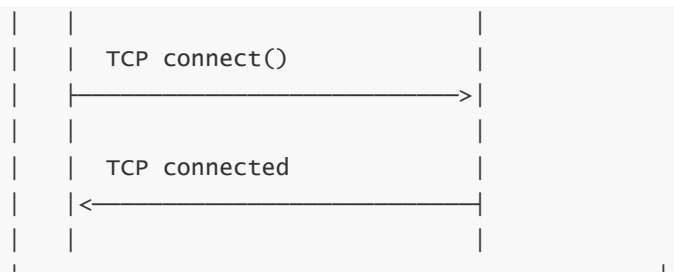
- 宇视设备是最复杂的设备类型, 原因:
  - 固定640字节buffer (协议要求, 不可更改)
  - 复杂的4步握手协议
  - 需要加密算法 (XOR加密random key)
  - 握手初步成功后还要发起一个接口的请求, Bridge请求。

这些特性使得宇视策略成为最复杂的策略实现。

- 完整握手流程

- 阶段1: TCP连接





阶段2: ReleaseRandomKey (服务器发送随机密钥)



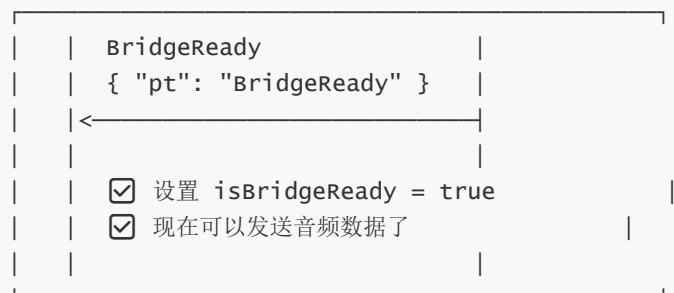
阶段3: RequireSession (客户端请求会话)



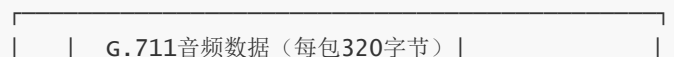
阶段4: SessionAck (服务器确认会话)

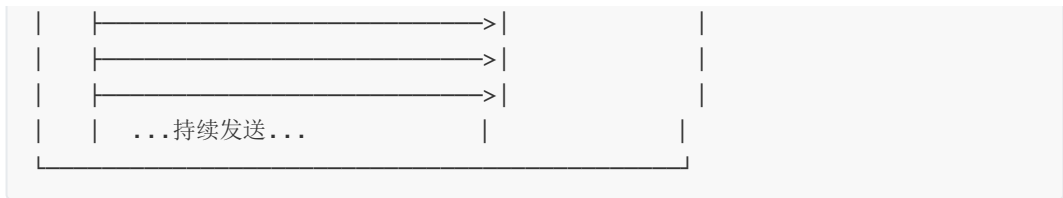


阶段5: BridgeReady (准备就绪)



阶段6: 发送音频数据





## 5.3 科达设备策略 KedaStrategy

- 特点:
  - 标准514字节buffer
  - 连接成功后立即发送协议头
  - 协议格式: `POST /postAudio?streamKey=xxx RS/1.1\r\n...`

## 5.4 自研NVR策略 WdzNvrStrategy

- 特点:
  - 标准514字节buffer
  - 模拟HTTP POST请求
  - Content-Length设置为99999999 (持续发送)

## 5.5 策略工厂 DeviceStrategyFactory

- 工厂模式如下:
 

```
object DeviceStrategyFactory {
 @JvmStatic
 fun createStrategy(deviceType: DeviceType): IDeviceStrategy {
 return when (deviceType) {
 DeviceType.WDZ_NVR -> WdzNvrStrategy()
 DeviceType.YUSHI -> YuShiStrategy()
 DeviceType.KEDA -> KedaStrategy()
 DeviceType.EZVIZ -> EzvizStrategy()
 }
 }
}
```

- 扩展新设备只需3步:
  - 添加枚举类型
  - 实现IDeviceStrategy接口
  - 在工厂中注册

# 06.核心组件层

## 6.1 多线程流水线设计

- 如果用单线程:
  - [采集 → 编码 → 发送] 串行执行  
 问题: 每个环节都阻塞, 总延迟 = 40ms(采集) + 5ms(编码) + 20ms(网络) = 65ms  
 结果: AudioRecord内部buffer会溢出, 丢帧严重
- 如果用多线程 (采集+编码合并) :

- [Thread1: 采集→编码] → [Thread2: 发送]  
问题: 编码5ms会延迟下一次采集, 40ms节奏被打乱  
结果: 仍然会丢帧

- **三线程流水线:**

- [Thread1: 采集40ms] → Queue1 → [Thread2: 编码5ms] → Queue2 → [Thread3: 发送20ms]  
优势: 每个线程独立运行, 队列解耦, 可以并行处理

- **本质: 将阻塞操作解耦, 用队列缓冲, 实现流水线并行**

## 6.2 队列选型

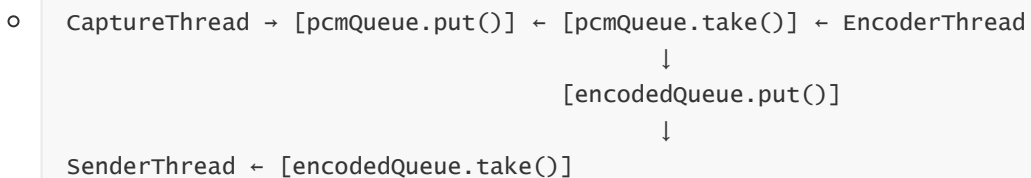
为什么用 `LinkedBlockingQueue` 而不是 `ArrayBlockingQueue`?

- 差异对比

| 特性   | <code>ArrayBlockingQueue</code> | <code>LinkedBlockingQueue</code> |
|------|---------------------------------|----------------------------------|
| 底层结构 | 数组                              | 链表                               |
| 锁机制  | 单把锁 (putLock == takeLock)       | 两把锁 (putLock ≠ takeLock)         |
| 并发性  | put和take互斥                      | put和take可以并发                     |
| 内存   | 预分配数组                           | 按需分配节点                           |

- 为什么两把锁至关重要

- 在我们的三线程流水线中有两个队列



- 如果用 `ArrayBlockingQueue` (单把锁) :

- `pcmQueue` 的put (采集) 和take (编码) 互斥, 吞吐量减半
- `encodedQueue` 同理

为什么队列的容量设计 都是20?

- 一帧是40ms, 20帧缓冲800ms, 可以支撑足够的网络抖动了
- 计算过程

- 采样率: 8000Hz  
位深: 16bit = 2字节  
声道: 单声道  
帧大小: 640字节  
帧时长:  $640 / (8000 * 2) = 0.04\text{秒} = 40\text{ms}$   
  
队列容量 20 = 20帧 = 800ms缓冲

- **设计权衡:**

- **实时性要求:** 对讲延迟最好 < 1秒

- **网络容错**：需要缓冲短暂的网络抖动（200-500ms）
- **最终选择**：800ms = 实时性和容错性的平衡点

### 6.3 ByteBufferPool 对象池

为什么懒加载而不是预分配？

- 错误的理解（预分配）

- ```
// ❌ 不是这样的：
init {
    repeat(10) { pool.offer(ByteBuffer.allocate(640)) }
}
```

- 实际的实现（懒加载）

- ```
fun acquire(): ByteBuffer {
 val buffer = pool.poll() // 尝试从池中取
 return if (buffer != null) {
 buffer.clear() // 复用
 buffer
 } else {
 createCount++ // 池为空才创建
 ByteBuffer.allocate(bufferSize)
 }
}
```

- 为什么懒加载更聪明

| 对比   | 预分配10个       | 懒加载                 |
|------|--------------|---------------------|
| 启动速度 | 慢（立即分配6.4KB） | 快（按需分配）             |
| 内存效率 | 浪费（实际只需3个）   | 高效（只创建需要的）          |
| 扩展性  | 差（固定10个）     | 好（自动扩展到maxPoolSize） |
| 适应性  | 差（无法感知实际并发）  | 强（根据实际负载调整）         |

- 真实场景分析

- 启动阶段（前200ms）：
  - CaptureThread还没开始采集
  - 池为空，无需预分配
  - 节省内存，启动更快
- 稳定运行阶段（200ms后）：
  - 3个线程并发：
    - \* CaptureThread持有1个buffer（正在read）
    - \* EncoderThread持有1个buffer（正在encode）
    - \* 队列中2-3个buffer
  - 总共需要4-5个buffer
- 高峰阶段（网络卡顿）：
  - 队列堆积，可能需要8-10个buffer
  - 懒加载会自动创建到这个数量
- 恢复阶段：





```

 if (offered) {
 // 成功放入队列，不回收buffer（由编码线程负责回收）
 buffer = null
 } else {
 // 队列满，丢弃数据，buffer会在finally中回收
 pcmDropCount++ // TODO-2: 统计丢包
 Log.w(TAG, "PCM queue full, dropping frame")
 }
 } else if (bytesRead < 0) {
 // 读取失败，buffer会在finally中回收
 Log.e(TAG, "AudioSource read error: $bytesRead")
 listener?.onError(Exception("AudioRecord read
error: $bytesRead"))
 }
 // 如果 bytesRead == 0，不做处理，buffer会在finally中回收

 } catch (e: Exception) {
 if (!shouldStop) {
 Log.e(TAG, "Error in capture thread", e)
 listener?.onError(e)
 }
 } finally {
 // 确保buffer一定被回收（除非已经放入队列）
 buffer?.let { bufferPool.release(it) }
 }
}

Log.d(TAG, "CaptureThread stopped")
}, "AudioCaptureThread")

captureThread?.start()
}

```

- startEncoderThread编码线程

- pcmQueue.poll从缓冲队列中取出数据，然后进行编码，将编码后的结果放入到编码队列

```

 /**
 * 启动编码线程
 * 职责：从 pcmQueue 取数据，编码后放入 encodedQueue
 */
 private fun startEncoderThread() {
 encoderThread = Thread({
 Log.d(TAG, "EncoderThread started")

 while (!shouldStop && !Thread.currentThread().isInterrupted)
 {
 try {
 // 阻塞poll，等待数据（超时100ms）
 val buffer =
 pcmQueue.poll(AudioConfig.QUEUE_POLL_TIMEOUT_MS, TimeUnit.MILLISECONDS)

 if (buffer != null) {
 // 转换为ByteArray
 val pcmData = ByteArray(buffer.remaining())
 buffer.get(pcmData)

 // 回收buffer

```

```

 bufferPool.release(buffer)

 // 编码
 val encodedData = encoder?.encode(pcmData)

 if (encodedData != null &&
encodedData.isNotEmpty()) {
 // 放入编码队列
 val offered =
encodedQueue.offer(encodedData)

 if (!offered) {
 encodedDropCount++ // TODO-2: 统计丢包
 Log.w(TAG, "Encoded queue full, dropping
frame")
 }
 } else {
 Log.w(TAG, "Encoder returned null or empty
data")
 }
 }

 } catch (e: InterruptedException) {
 // 正常中断
 break
 } catch (e: Exception) {
 if (!shouldStop) {
 Log.e(TAG, "Error in encoder thread", e)
 listener?.onError(e)
 }
 }
}

Log.d(TAG, "EncoderThread stopped")
}, "AudioEncoderThread")

encoderThread?.start()
}

```

- startSenderThread发送线程

- 队列中取出数据，然后发送

```

◦ /**
 * 启动发送线程
 * 职责：从 encodedQueue 取数据，通过 channel 发送
 */
 private fun startSenderThread() {
 senderThread = Thread({
 Log.d(TAG, "SenderThread started")

 while (!shouldStop && !Thread.currentThread().isInterrupted)
{
 try {
 // 阻塞poll，等待数据（超时100ms）
 val encodedData =
encodedQueue.poll(AudioConfig.QUEUE_POLL_TIMEOUT_MS,
TimeUnit.MILLISECONDS)

```

```

 if (encodedData != null) {
 // 检查设备是否允许发送（如字幕设备需要先握手）
 val canSend = deviceStrategy?.canSendAudio() ?:
true

 if (canSend) {
 // TODO-4: 首帧发送日志
 if (!firstAudioSent) {
 Log.i(TAG, "First audio packet sent,
size=${encodedData.size} bytes")
 firstAudioSent = true
 }

 // 发送数据
 channel?.send(encodedData)

 // TODO-1: 统计发送成功
 sendSuccessCount++
 } else {
 // 设备未就绪，丢弃数据（握手未完成）
 sendDropCount++ // TODO-1: 统计丢包

 // 每秒输出一警告
 if (Math.random() < 0.025) {
 Log.w(TAG, "Device not ready, dropping
${encodedData.size} bytes")
 }
 }

 // TODO-1: 定期输出统计（每5秒）
 val now = System.currentTimeMillis()
 if (now - lastStatTime >= 5000) {
 val totalSent = sendSuccessCount +
sendDropCount

 val successRate = if (totalSent > 0) {
 (sendSuccessCount * 100.0 / totalSent)
 } else {
 0.0
 }

 Log.i(TAG, "Audio Stats [5s]:
Sent=${sendSuccessCount}, Dropped=${sendDropCount}, " +
 "PCM_Drop=${pcmDropCount},
Encoded_Drop=${encodedDropCount}, " +
 "Success=${"%0.1f".format(successRate)}%")

 lastStatTime = now
 }
 }

 } catch (e: InterruptedException) {
 // 正常中断
 break
 } catch (e: Exception) {
 if (!shouldStop) {
 Log.e(TAG, "Error in sender thread", e)

```

```
 listener?.onError(e)
 }
}

Log.d(TAG, "SenderThread stopped")
}, "AudioSenderThread")

senderThread?.start()
}
```

## 05.深度思考

---

### 5.1 关键问题探究

### 5.2 设计对比

## 06.实践验证

---

### 6.1 行为验证代码

### 6.2 性能测试

## 07.应用场景

---

### 7.1 最佳实践

### 7.2 使用禁忌

## 08.总结提炼

---

### 8.1 核心收获

### 8.2 知识图谱

## 8.3 延伸思考

## 09.参考资料

---

- 1.
- 2.
- 3.

## 其他介绍

---

### 01.关于我的博客

- csdn: [http://my.csdn.net/qq\\_35829566](http://my.csdn.net/qq_35829566)
- 掘金: <https://juejin.im/user/499639464759898>
- github: <https://github.com/jjjjjjava>
- 邮箱: [[934137388@qq.com](mailto:934137388@qq.com)]